Estruturas de Dados Básicas II

Prof. Dr. Matheus da Silva Menezes



UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Continuando...

Estruturas de Dados Básicas II

COMPLEXIDADE COMPUTACIONAL

Relembrando...

Porque estudar complexidade de algoritmos?

1. Eficiência e Eficácia
2. Adequação à capacidade de processamento do equipamento
3. Viabilidade do código

E se..
Implementar o algoritmo, aplicar a várias instâncias do problema e medir o tempo?

Do que depende o tempo de processamento de um algoritmo?

1. Equipamento utilizado
2. Tamanho do Problema
3. Tipo de Estrutura de dados
4. Compilador/Linguagem/SO
5. Habilidade do programador

[...]

Análise de Algoritmos

Dado um modelo abstrato de computador (será visto adiante), Quantificar os recursos computacionais necessários à solução de um problema (todos os casos) como função do tamanho das entradas das instâncias

Como recursos entendemos a memória, o tempo de processamento, a natureza das operações, etc

O tamanho de uma instância corresponde ao número de bits necessários para codificá-la em um computador (entrada)

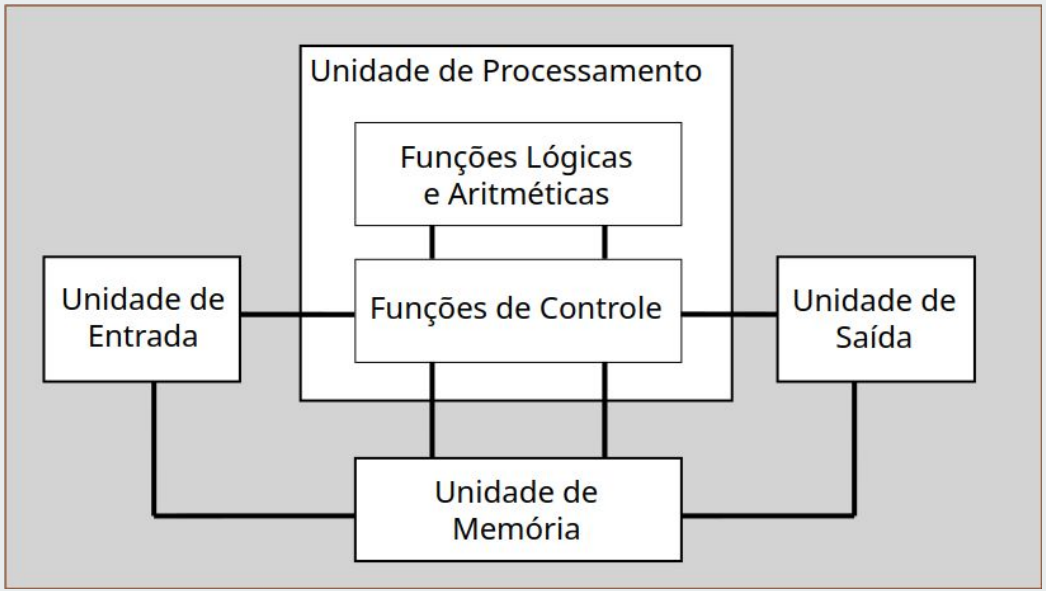
$x = 5 \leftarrow 101$ $n = 3$ bits

$y = 5.000.000 \leftarrow$ $n > 3$ bits

A análise do algoritmo não deve depender dos recursos de hardware utilizados



Modelo de Computação RAM



Operações:

- Entrada e Saída
- Operações Aritméticas
- Operações Lógicas
- Operações Relacionais
- Movimentação de Dados

Definição

O tempo de processamento de um algoritmo é a quantidade de passos básicos executados por ele sobre uma instância de entrada

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

$n = |A|$

```
somaMatriz(A,B,n)
para i = 1 até n
    para j = 1 até n
        C[i,j] = A[i,j] + B[i,j]
retorne(C)
```

$n \times n = n^2$



Tamanho da entrada?

COMPLEXIDADE EM TEMPO

O modelo considera que cada instrução `I` possuirá um tempo associado `t(I)` para ser operacionalizada em RAM.

São processadas r_1 instruções do tipo I_1, r_2 instruções do tipo I_2, ..., r_m instruções do tipo I_m.

Então o tempo total de execução `T(n)` será dado por:

$$T(n) = \sum_{j=1}^{m} r_j t(I_j)$$

10 x Tempo

COMPLEXIDADE EM TEMPO

A função $t_A: \{0,1\}^* \rightarrow \mathbb{N}$ é dita complexidade do algoritmo **A** se, para toda entrada **x**, **A** termina em exatamente $t_A(x)$ passos.

Análises:

- Pior Caso
- Melhor Caso
- Caso Médio

Melhor Caso

Estamos interessados na dependência da complexidade como função do tamanho da entrada, tomando o valor mínimo de passos para entradas de tamanho relevante.

$n \to \infty$

Pior Caso

Estamos interessados na dependência da complexidade como função do tamanho da entrada, tomando o valor máximo de passos para entradas de tamanho relevante.

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

A[10] (n+1)

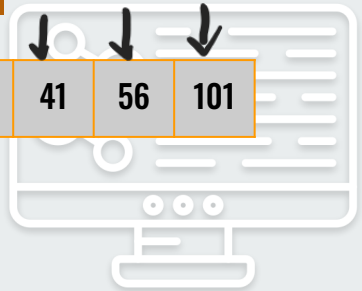
n = 10 x = 4

4	8	9	10	12	34	37	41	56	101

esq m dir

n = 10 x = 2

4	8	9	10	12	34	37	41	56	101



PSEUDOCÓDIGO

```
buscaLista(A,n,x)
1  i = 1                                   1
2  enquanto i<=n e A[i]!= x faça           4
3      i = i+1                             2
4  se i <= n e A[i]==x então               4
5      retorne i                           1
6  retorne -1                              1
```

	Melhor Caso	p vezes	Pior Caso
1	t	t	t
2	$4t$	$4t.(p)$	$(n+1)t + 3t(n)$
3		$2t(p-1)$	$2t(n)$
4	$4t$	$4t$	t
5	t	t	
6			t

$p = 3$

$6t.(3) + 4t = 22t$

Chave na primeira posição	Chave na posição $1 \leq p \leq n$	Não existe a chave procurada
$10t$	$6tp + 4t$	$6tn + 4t$

PSEUDOCÓDIGO

```
buscaBinaria(A,n,x)
1  esq = 1                                  1
2  dir = n                                  1
3  enquanto esq<dir faça                    1
4      m = ⌊(esq+dir)/2⌋                    4
5      se x > A[m] então                    2
6          esq = m+1                        2
7      senão
8          dir = m                          1
9  se A[esq]==x então                       2
10     retorne esq                          1
11 retorne -1                               1
```

Pior Caso

1	t
2	t
3	$t\,(R)$ ←
4	$4t\,(R-1)$
5	$2t\,(R-1)$
6	$2t$
7	$2t\,(R-1)$
8	$1t$
9	$2t$
10	t
11	

$5t + tR + 8t(R-1)$

$5t + tR + 8tR - 8t$

$9tR - 3t$

$9t \log_2 n - 3t$

RESUMO

		Melhor Caso	Pior Caso
1	Busca Linear	$10t$	$6tn + 4t$
2	Busca Binária	—	$9t \log_2 n - 3t$

$6t.(x) + b$

$a \cdot x + b$

$a \cdot \log(x) + b$

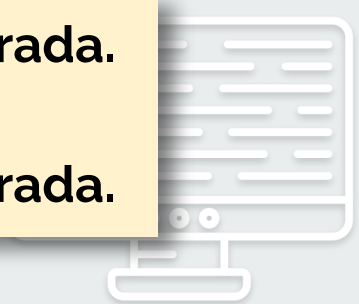
Complexidade Média

A questão que se coloca é qual seria a complexidade média para uma operação real em que esse algoritmo fosse aplicado a diversas instâncias do problema?

Na média, o que aconteceria?

Depende de como estão distribuídos os dados de entrada.

Precisamos conhecer a distribuição dos dados de entrada.



Complexidade Média

Precisamos conhecer:

1. Todas as possíveis entradas do algoritmo.
2. A probabilidade de ocorrência de cada uma delas.
3. O tempo de processamento de cada uma.

Qual a probabilidade de uma entrada E ocorrer?

Qual o tempo de processamento de E?



Complexidade Média

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

Busca com Sucesso

Casos em que a chave de busca **x** é igual a chave **A[i]**, **i = 1,...,n.**

Para encontrar a chave **x = A[i]**, o algoritmo faz **i** comparações.

E[i] representa o caso de sucesso onde **x = A[i]**.

Portanto **t(E[i])= i**



Número de entradas de sucesso = n

Complexidade Média

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

Busca sem Sucesso

Casos em que a chave de busca **x** não se encontra na lista.

Para concluir que a chave **x** não está na lista, o algoritmo faz **n** comparações.

R representa o caso de não sucesso.

Portanto **t(R)= n**

Obs. Os valores que resultam na busca sem sucesso podem ser agrupados em um único conjunto: **X ≠ A[i]**

Número de entradas sem sucesso = 1

Complexidade Média

Qual a probabilidade de cada uma das entradas ocorrerem?

Supor distribuição uniforme:

Probabilidade de `x` estar na posição `i` é `1/n`

$$CM = \sum_{E_i \in S} p(E_i)\, t(E_i)$$

- **S** é o conjunto de todas as entradas possíveis
- **E_i** é um elemento de **S**
- **$p(E_i)$** é a probabilidade de ocorrência da entrada **E_i**
- **$t(E_i)$** é o tempo requerido pela entrada **E_i** para ser processada

Complexidade Média

Considerando que a probabilidade de $x \in S$ é modelada através de uma distribuição uniforme, e que seu valor será igual a q, então:

Probabilidade de sucesso = q

Probabilidade de insucesso = $1-q$

Então, considerando distribuição uniforme e que temos n elementos em S, a probabilidade de x estar na i-ésima posição da lista S é q/n.

Complexidade Média

$$CM = \sum_{i=1}^{n} \left(p(E_i) t(E_i) \right) + p(R) t(R)$$

$$t(E_i) = i$$

$$t(R) = n$$

$$p(E_i) = \frac{q}{n}$$

$$p(R) = 1 - q$$



Complexidade Média

$$CM = \sum_{i=1}^{n} \left(p(E_i) t(E_i) \right) + p(R) t(R)$$

$p(E_i) = \frac{q}{n}$ $t(E_i) = i$ $p(R) = 1 - q$ $t(R) = n$

$$CM = \sum_{i=1}^{n} \frac{q}{n} i + (1 - q) n$$



Complexidade Média

$$CM = \sum_{i=1}^{n} \frac{q}{n} i + (1-q)n$$

$$CM = n - \frac{q}{2}(n-1)$$

$$\frac{q}{n} \sum_{i=1}^{n}(i) + (1-q) \cdot n$$

$$\frac{q}{n} (n+1) \cdot \frac{n}{2} + (1-q) \cdot n$$

$$\frac{q(n+1)}{2} + (1-q)n$$

$$\frac{qn}{2} + \frac{q}{2} + n - qn$$

$$\frac{q}{2}(n+1) +$$

$$-\frac{qn}{2} + \frac{q}{2} + n \therefore \boxed{n + \frac{q}{2}(1-n)}$$



Complexidade Média

$$CM = n - \frac{q}{2}(n-1)$$

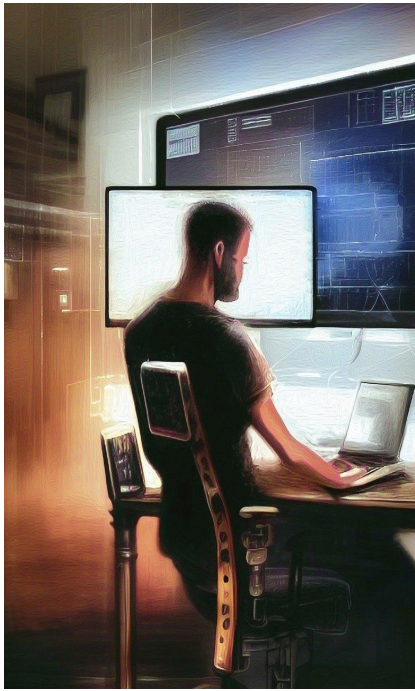
- q=1 → CM = (n+1)/2
- q=0 → CM = n
- q=1/2 → CM = (3n+1)/4

	Melhor Caso	Pior Caso
1	Busca Linear	

Exercícios...

1. A complexidade muda se a lista estiver ordenada e utilizarmos o algoritmo2?
2. Qual a complexidade média no caso do algoritmo2?
3. Qual seria o melhor caso para a busca binária?





Dúvidas?!



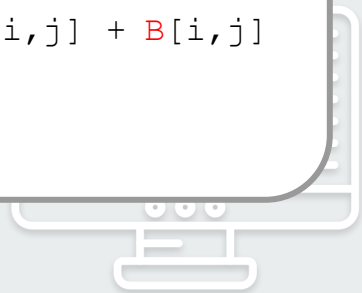
Definição

O tempo de processamento de um algoritmo é a quantidade de passos básicos executados por ele sobre uma instância de entrada

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

```
ePar(x)
if x % 2 == 0
   retorne(true)
retorne(false)
```

```
somaMatriz(A,B,n)
para i = 1 até n
   para j = 1 até n
      C[i,j] = A[i,j] + B[i,j]
   retorne(C)
```



Tamanho da entrada?

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
1  i = 1
2  enquanto i<=n e A[i]!= x faça
3      i = i+1
4  se i <= n e A[i]==x então
5      retorne i
6  retorne -1
```

	Melhor Caso	Pior Caso
1		
2		
3		
4		
5		
6		

n = 4 x = 56

4	8	9	10	12	34	37	41	56	101

n = 10 x = 2

4	8	9	10	12	34	37	41	56	101